DATA STRUCTURES

UNIT 3: LINKED LIST

POLYNOMIAL CALCULATOR

Polynomial Calculator Using Singly Linked List

ORIGINAL DOCUMENTATION

Subject	Data Structures
Unit	Unit 3 – Linked List
Project	Polynomial Calculator
Documentation	Original
Programming Language	C++

Prepared for Second-Year Engineering – Data Structures Project Documentation

1. Introduction

A polynomial is a mathematical expression containing terms with coefficients and non-negative integer exponents, such as $4x^3 + 2x^2 – 5$. A polynomial calculator stores each term as a node in a singly linked list. Each node contains a coefficient, an exponent, and a pointer to the next node. This project demonstrates how linked lists can store polynomial terms dynamically and support basic polynomial operations.

2. Problem Statement

Develop a C++ program that represents polynomials using a singly linked list and performs basic operations such as addition, subtraction, multiplication, and display. The program should combine terms having the same exponent and present the result in descending order of exponents.

3. Objectives

- To represent each polynomial term using a node containing coefficient and exponent.
- To understand dynamic memory allocation and links between nodes.
- To perform polynomial addition and subtraction by combining terms with equal exponents.
- To multiply two polynomials and simplify the resulting expression.
- To display polynomial terms in descending order of exponents.

4. Data Structure Used

Singly Linked List (SLL): Each node stores a coefficient, exponent, and pointer to the next node. The last node points to NULL. Nodes can be created dynamically, so the number of polynomial terms does not need to be fixed in advance.

Field	Purpose
coefficient	Stores the numerical coefficient of a term.
exponent	Stores the power of x.
next	Points to the next term node.

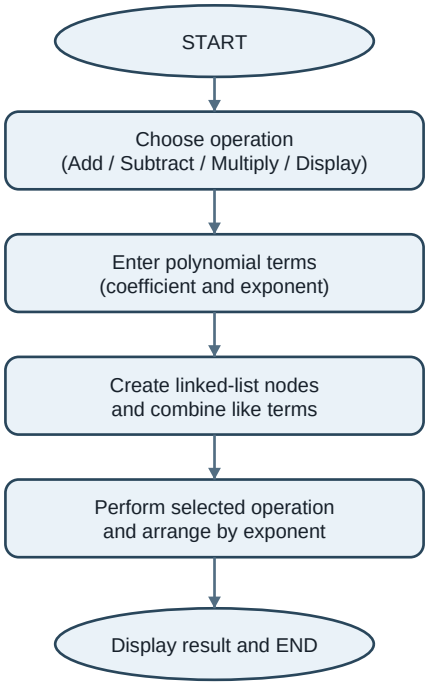
5. Algorithm

Algorithm: Polynomial Calculator

1. Start the program.
2. Choose the linked-list type (modified version only).
3. Read the terms of Polynomial P and Polynomial Q as coefficient–exponent pairs.
4. Create a node for each term and insert it into the selected linked list.
5. Arrange terms by exponent and combine nodes having equal exponents.
6. Display the menu: Add, Subtract, Multiply, or Display.
7. For addition/subtraction, combine coefficients of terms with equal exponents.
8. For multiplication, multiply every term of P by every term of Q; add exponents and multiply coefficients.
9. Combine like terms in the result and remove zero-coefficient terms.
10. Display the simplified polynomial and stop, or return to the menu.

6. Flowchart

The flowchart below shows the overall processing sequence. In the modified version, list-type selection is included before term creation.



7. Working of the Project

The user enters each term as a coefficient and exponent. The program creates a node for each term and links it to the next node. During addition or subtraction, terms with the same exponent are matched and their coefficients are added or subtracted. During multiplication, each term of the first polynomial is multiplied by every term of the second polynomial; coefficients are multiplied and exponents are added. The result is simplified by combining like terms and then displayed in descending exponent order.

8. Programming Language and Concepts

- **Language:** C++.
- **Structures/classes:** Represent polynomial nodes and list operations.
- **Pointers:** Connect nodes and support dynamic list traversal.
- **Dynamic memory allocation:** Create nodes as required.
- **Functions:** Separate input, insertion, display, simplification, and arithmetic operations.
- **Loops and conditional statements:** Traverse lists and match exponents.
- **Arithmetic rules:** For multiplication, (a x■)(b x■) = ab x■■■.

9. Advantages

- Memory is allocated dynamically according to the number of terms.
- Polynomial size is not restricted to a fixed array length.
- Like terms can be combined to simplify the result.
- The program is modular and can be extended with additional operations.

10. Applications

- Computer algebra and symbolic mathematical calculations.
- Educational demonstration of linked lists and dynamic memory.
- Polynomial manipulation in engineering and scientific computations.
- Practice in pointer-based data structures and modular C++ programming.

11. Conclusion

The project shows how a singly linked list can represent polynomial terms and perform addition, subtraction, and multiplication efficiently for a variable number of terms. It strengthens understanding of pointers, dynamic memory allocation, traversal, and basic data-structure operations.

12. Sample Input and Output

Input	Polynomial P = $3x^2 + 2x + 1$ Polynomial Q = $x + 4$
Addition	P + Q = $3x^2 + 3x + 5$
Subtraction	P – Q = $3x^2 + x – 3$
Multiplication	P × Q = $3x^3 + 14x^2 + 9x + 4$

Note: The sample results are illustrative. The final C++ implementation should validate input, handle empty polynomials, and avoid duplicate or zero-coefficient terms.